Comprehensive Python for Data Science

Jupyter Architecture, Memory Management, and Advanced Data Structures

Course: Data Analysis & AI Fundamentals
MEDIACREST TRAINING COLLEGE

Instructor: Victor Wandera Lumumba

Contents

1 The Python Ecosystem & Environment Setup

Before writing code, it is crucial to understand the environment in which Python operates. Python is an **interpreted, high-level, object-oriented** programming language. Unlike compiled languages (C++, Java), Python code is executed line-by-line by the Python interpreter, making it highly interactive but slightly slower in raw execution speed (which is mitigated by C-optimized libraries like NumPy).

1.1 Package Management & Virtual Environments

In professional data science, you never install packages globally on your operating system. You use **Virtual Environments**.

- **Why?** Different projects require different versions of libraries (e.g., Project A needs Pandas 1.5, Project B needs Pandas 2.0). Virtual environments isolate dependencies.
- **Tools:** `venv` (built-in), `conda` (Anaconda/Miniconda - highly recommended for data science as it handles non-Python C-dependencies seamlessly), or `poetry`.

```
# Creating a virtual environment using venv
python -m venv my_data_env

# Activating it (Windows)
my_data_env\Scripts\activate

# Activating it (Mac/Linux)
source my_data_env/bin/activate

# Installing packages safely inside the environment
pip install pandas numpy matplotlib jupyter
```

2 Mastering Jupyter Notebooks

Jupyter is more than just a text editor; it is a web-based interactive computing platform.

2.1 Architecture

A Jupyter Notebook application consists of three main components:

1. **The Web Browser:** The frontend interface where you write code and view outputs.
2. **The Jupyter Server:** Handles file management, authentication, and routing.
3. **The Kernel:** The actual engine (e.g., IPython) that executes the code. *Crucial concept:* The kernel maintains the **state** (variables in memory) across all cells. If you run Cell 1 (`x = 5`) and then Cell 2 (`print(x)`), Cell 2 works because the kernel remembers `x`.

2.2 IPython Magic Commands

Jupyter includes "magic commands" designed to solve common data science problems. They start with `%` (line magic) or `%%` (cell magic).

```
# Time how long a line of code takes to execute
%timeit sum(range(10000))

# Time an entire cell
%%timeit
```

```
total = 0
for i in range(10000):
    total += i

# Show all output in a cell (by default, Jupyter only shows the last
    output)
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"

# System shell commands (preceded by !)
!pip list
!ls -l
```

3 Variables, Memory, and Object References

This is where most beginners struggle. The standard "variable is a bucket" analogy is **wrong** in Python.

Under the Hood:

In Python, variables are **labels** or **name tags** attached to objects in memory, not containers holding data. When you write `x = 10`, Python creates an integer object `10` in memory, and attaches the label `x` to it.

3.1 Identity vs. Equality

- `==` checks if the **values** of two objects are the same.
- `is` checks if two variables point to the exact **same object in memory** (same memory address).

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)  # True (Values are identical)
print(a is b)  # False (They are two different list objects in memory)
print(a is c)  # True (c is just another label for the exact same
    object as a)

# Checking memory addresses
print(id(a))   # e.g., 140409234567890
print(id(c))   # e.g., 140409234567890 (Same as 'a')
```

3.2 Mutability

Objects in Python are either **mutable** (can be changed after creation) or **immutable** (cannot be changed).

- **Immutable:** `int`, `float`, `str`, `bool`, `tuple`. If you "modify" an immutable object, Python actually creates a **brand new object** in memory.
- **Mutable:** `list`, `dict`, `set`. You can change their contents without changing their memory identity.

Common Pitfall: The Mutable Default Argument Trap

Never use a mutable object (like a list) as a default argument in a function. The default is evaluated *only once* when the function is defined, not each time it is called.

```
# BAD PRACTICE
def add_patient(patient_list=[]):
    patient_list.append("New Patient")
    return patient_list

print(add_patient()) # ['New Patient']
print(add_patient()) # ['New Patient', 'New Patient'] -> Bug!
```

Solution: Use `None` as the default and initialize inside the function.

4 Deep Dive into Core Data Types

4.1 Numeric Types & Float Precision

Python handles integers (`int`) with arbitrary precision (they won't overflow). However, floats (`float`) are implemented using C's `double` (64-bit IEEE 754), which means they suffer from precision issues.

```
# The classic floating-point arithmetic trap
print(0.1 + 0.2)          # Output: 0.30000000000000004
print(0.1 + 0.2 == 0.3)   # Output: False!

# How to handle it in Data Science:
import math
print(math.isclose(0.1 + 0.2, 0.3)) # True

# For financial data, NEVER use floats. Use the decimal module.
from decimal import Decimal
print(Decimal('0.1') + Decimal('0.2') == Decimal('0.3')) # True
```

4.2 Advanced String Manipulation

Strings are immutable sequences of Unicode characters.

4.2.1 F-Strings (Formatted String Literals)

Introduced in Python 3.6, f-strings are the most efficient and readable way to format strings.

```
patient_name = "Alice"
age = 28
bp = 120.456

# Basic formatting
print(f"Patient {patient_name} is {age} years old.")

# Mathematical expressions inside f-strings
print(f"Next year, she will be {age + 1}.")

# Formatting numbers (padding, decimals)
print(f"Blood Pressure: {bp:.2f}")  # Rounds to 2 decimal places:
    120.46
print(f"Age padded: {age:05}")      # Pads with zeros: 00028
```

```

# Alignment
print(f"{'Name':<10} | {'Age':>5}") # Left align Name, Right align Age
```

4.3 Booleans and "Truth Value Testing"

In Python, every object has an inherent boolean value (`True` or `False`).

- **Falsy values:** `False`, `0`, `0.0`, `""` (empty string), `[]` (empty list), {} (empty dict), `None`.
- **Truthy values:** Almost everything else.

```
data = []

# Instead of writing: if len(data) == 0:
# Pythonic way:
if not data:
    print("The dataset is empty!")
```

5 Advanced Data Structures

5.1 Lists: Comprehensions and Copying

Lists are mutable, ordered sequences. The most "Pythonic" feature is the **List Comprehension**, which replaces standard `for` loops for creating lists.

```
# Standard loop
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x**2)

# List Comprehension (Faster and more readable)
squares = [x**2 for x in range(10) if x % 2 == 0]

# Nested list comprehension (Flattening a matrix)
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [num for row in matrix for num in row]
# Result: [1, 2, 3, 4, 5, 6]
```

5.1.1 Shallow vs. Deep Copy

Because lists are mutable, copying them requires care.

```
import copy

original = [[1, 2], [3, 4]]

# Shallow copy (copies the outer list, but inner lists are still
    references)
shallow = original.copy()
shallow[0][0] = 99
print(original[0][0]) # 99! The original was modified.

# Deep copy (recursively copies everything)
deep = copy.deepcopy(original)
deep[0][0] = 100
```

```
print(original[0][0]) # 99. The original is safe.
```

5.2 Tuples: The Immutable Sibling

Tuples (()) are like lists, but **immutable**.

- **Why use them?** They are more memory-efficient and slightly faster than lists.
- **Crucial use case:** Because they are immutable, tuples can be used as **dictionary keys** or elements in a **set**. Lists cannot.

```
# Tuple unpacking (Highly used in Data Science / Pandas)
coordinates = (40.7128, -74.0060)
lat, lon = coordinates

# Swapping variables elegantly
a, b = 1, 2
a, b = b, a  # No temporary variable needed!
```

5.3 Dictionaries: Hash Tables Under the Hood

Dictionaries ({}) store key-value pairs.

Under the Hood:

Python dictionaries are implemented as **Hash Tables**. When you insert a key, Python runs it through a hash function to compute an index in an array. This is why dictionary lookups are **O(1)** (constant time) on average, making them incredibly fast even for millions of records. The trade-off is that they consume more memory.

5.3.1 Advanced Dictionary Methods

```
patient = {"name": "John", "age": 30}

# .get() prevents KeyError if the key doesn't exist
print(patient.get("blood_type", "Unknown")) # Returns "Unknown"

# .update() merges dictionaries
patient.update({"age": 31, "city": "Nairobi"})

# Dictionary Comprehension
squares_dict = {x: x**2 for x in range(5)}
# Result: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Grouping data (A common data wrangling task)
data = [("Malaria", 10), ("Flu", 5), ("Malaria", 15)]
grouped = {}
for disease, count in data:
    grouped.setdefault(disease, []).append(count)
# Result: {'Malaria': [10, 15], 'Flu': [5]}
```

5.4 Sets: Mathematical Collections

Sets ({}) are unordered collections of **unique** elements. Like dictionaries, they are implemented using hash tables.

```python
# Instant deduplication
ages = [22, 25, 22, 30, 25, 28]
unique_ages = set(ages) # {22, 25, 30, 28}

# Extremely fast membership testing (O(1) vs O(N) for lists)
valid_ids = set(range(1000000))
print(999999 in valid_ids) # Instantaneous

# Mathematical Set Operations
group_A = {"Alice", "Bob", "Charlie"}
group_B = {"Charlie", "David", "Eve"}

print(group_A | group_B)  # Union: All unique elements
print(group_A & group_B)  # Intersection: Elements in both
print(group_A - group_B)  # Difference: In A but not in B
```

6 Control Flow & Functions

6.1 Itertools: Supercharging Loops

The built-in `itertools` library provides highly optimized looping mechanisms.

```python
import itertools

# Enumerate (Get index and value simultaneously)
diseases = ["Malaria", "Flu", "Dengue"]
for index, disease in enumerate(diseases, start=1):
    print(f"{index}. {disease}")

# Zip (Iterate over multiple lists in parallel)
names = ["Alice", "Bob"]
ages = [25, 30]
for name, age in zip(names, ages):
    print(f"{name} is {age}")

# Combinations and Permutations
print(list(itertools.combinations([1, 2, 3], 2)))
# [(1, 2), (1, 3), (2, 3)]
```

6.2 Functions, Scope, and Lambda

Functions are defined using `def`. Python uses the **LEGB** rule for variable scope: **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in.

```python
# *args and **kwargs allow flexible argument passing
def analyze_data(*args, **kwargs):
    print(f"Analyzing {len(args)} datasets.")
    print(f"Parameters: {kwargs}")

analyze_data("sales.csv", "hr.csv", normalize=True, threshold=0.05)

# Lambda functions (Anonymous, single-expression functions)
# Highly useful in Pandas .apply() methods
square = lambda x: x**2
print(square(5))

```

```python
# Sorting complex data structures
patients = [("Alice", 30), ("Bob", 22), ("Charlie", 28)]
# Sort by age (the second element in the tuple)
patients.sort(key=lambda p: p[1])
```

7 Error Handling & Debugging

In data cleaning, you will encounter messy data that breaks your code. Robust pipelines use `try/except` blocks to handle exceptions gracefully without crashing the entire program.

```python
def parse_age(age_str):
    try:
        # Attempt to convert string to integer
        age = int(age_str)
        if age < 0:
            raise ValueError("Age cannot be negative.")
        return age
    except ValueError as e:
        # Catch specific errors
        print(f"Warning: Invalid age format '{age_str}'. Error: {e}")
        return None
    except Exception as e:
        # Catch-all for unexpected errors
        print(f"An unexpected error occurred: {e}")
        return None
    finally:
        # This block ALWAYS executes (useful for closing files/
    connections)
        print("Attempted to parse age.")

print(parse_age("25"))    # Returns 25
print(parse_age("Twenty"))# Warning... Returns None
print(parse_age("-5"))    # Warning... Returns None
```

8 Modern Python Practices: Type Hinting

While Python is dynamically typed, modern Python (3.5+) supports **Type Hinting**. This doesn't enforce types at runtime, but allows IDEs (like VS Code) and static type checkers (like `mypy`) to catch bugs before you run the code. It is highly recommended for collaborative data science projects.

```python
from typing import List, Dict, Optional, Tuple

# Defining types for variables and function signatures
def calculate_bmi(weight_kg: float, height_m: float) -> float:
    return weight_kg / (height_m ** 2)

def get_patient_data(patient_id: int) -> Optional[Dict[str, any]]:
    """Returns a dictionary of patient data, or None if not found."""
    database = {1: {"name": "Alice", "age": 30}, 2: {"name": "Bob", "
    age": 25}}
    return database.get(patient_id)

# Complex types
def process_records(data: List[Tuple[str, float]]) -> Dict[str, List[
    float]]:
```

```
    pass
```

9 Comprehensive Practice Exercises

9.1 Exercise 1: Memory & Mutability

Predict the output of the following code and explain *why* based on Python's memory model.

```
data_a = [1, 2, [3, 4]]
data_b = data_a.copy()
data_b[0] = 99
data_b[2][0] = 300

print(data_a)
print(data_b)
```

9.2 Exercise 2: Data Wrangling with Dictionaries

You are given a raw list of strings representing clinical visits: `["PatientA:120", "PatientB:130", "PatientA:125", "PatientC:110", "PatientB:135"]`. Write a script using dictionaries to group the blood pressure readings by patient, resulting in: {`"PatientA": [120, 125], "PatientB": [130, 135], "PatientC": [110]`}. Then, calculate the average BP for each patient using a dictionary comprehension.

9.3 Exercise 3: Robust Parsing

Write a function `clean_financial_data(raw_list: List[str]) -> List[float]` that takes a list of strings like `["100.50", "N/A", "200", "error", "300.00"]`. *It should strip currency symbols, convert valid*

9.4 Exercise 4: Advanced Comprehensions

Given a matrix (list of lists) representing a grid of pixel intensities: `matrix = [[0, 255, 0], [255, 0, 255], [0, 255, 0]]` Write a single list comprehension that flattens the matrix and converts all values to binary (`1 if > 127, else 0`).

End of Comprehensive Notes. Master these fundamentals, and the transition to NumPy and Pandas will be seamless.